```
Cache-Control: no-cache

$
```

We see there is an empty response (no data after the `Cache-Control` line), but the request was successful because Rails has set the response to 200 OK. You can set the `:status` option on render to change this response. Rendering nothing can be useful for Ajax requests where all you want to send back to the browser is an acknowledgment that the request was completed.

You should probably be using the `head` method, discussed later in this guide, instead of `render :nothing`. This provides additional flexibility and makes it explicit that you're only generating HTTP headers.

2.2.2 Rendering an Action's View

If you want to render the view that corresponds to a different template within the same controller, you can use `render` with the name of the view:

```
def update
  @book = Book.find(params[:id])
  if @book.update(book_params)
    redirect_to(@book)
  else
    render "edit"
  end
end
```

If the call to `update` fails, calling the `update` action in this controller will render the `edit.html.erb`template belonging to the same controller.

If you prefer, you can use a symbol instead of a string to specify the action to render:

```
def update
  @book = Book.find(params[:id])
  if @book.update(book_params)
    redirect_to(@book)
  else
    render :edit
  end
end
```

2.2.3 Rendering an Action's Template from Another Controller

What if you want to render a template from an entirely different controller from the one that contains the action code? You can also do that with `render`, which accepts the full path (relative to `app/views`) of the template to render. For example, if you're running code in an `AdminProductsController` that lives in `app/controllers/admin`, you can render the results of an action to a template in `app/views/products` this way:

```
render "products/show"
```

Rails knows that this view belongs to a different controller because of the embedded slash character in the string. If you want to be explicit, you can use the `:template` option (which was required on Rails 2.2 and earlier):

```
render template: "products/show"
```

2.2.4 Rendering an Arbitrary File

The `render` method can also use a view that's entirely outside of your application (perhaps you're sharing views between two Rails applications):

```
render "/u/apps/warehouse_app/current/app/views/products/show"
```

Rails determines that this is a file render because of the leading slash character. To be explicit, you can use the `:file` option (which was required on Rails 2.2 and earlier):

```
render file: "/u/apps/warehouse_app/current/app/views/products/show"
```